QuarryOS

Quarry Fleet & Workforce Management System

BACKEND DEVELOPMENT SPECIFICATION

Version 1.0 | April 2026 | Internal

6	7	30+	REST
App Pages	DB Tables	API Endpoints	JSON API

1. Project Overview

QuarryOS is a full-stack operations management system for a quarry worksite. It manages vehicle fleets (trucks, pickups, minivans), worker assignments, job dispatch, return verification, financial tracking, and maintenance records.

1.1 System Purpose

Primary Users	Site managers / dispatchers who assign vehicles and workers daily
Secondary Users	Accountants who track worker earnings and vehicle expenses
Core Problem	No fixed vehicle per worker — assignments are made fresh each dispatch cycle
Key Constraint	Wages are auto-calculated per vehicle type and distance. Workers can drive any vehicle type.
Operating Context	Quarry site with intermittent connectivity — offline-tolerant design recommended

1.2 Tech Stack Recommendation

Backend Framework	Node.js + Express.js (or FastAPI / Django if Python preferred)
Database	PostgreSQL (relational, strong for financial reporting)
ORM	Prisma (Node) or SQLAlchemy (Python)
Authentication	JWT (JSON Web Tokens) — stateless, mobile-friendly
File Storage	Local filesystem or AWS S3 for any future photo uploads
API Style	RESTful JSON API — all responses wrapped in { success, data, error }
Hosting	Any VPS (DigitalOcean, Render, Railway) or self-hosted LAN server

Recommendation for a small team
Node.js + Express + PostgreSQL + Prisma is the fastest path to production.
Railway.app gives a free PostgreSQL + Node hosting in under 10 minutes.
If the team prefers Python, FastAPI + PostgreSQL + SQLAlchemy is equally solid.

2. Database Schema

All tables use UUID primary keys for portability. Timestamps use UTC. Soft deletes (deleted_at) are recommended for workers and vehicles to preserve historical data.

2.1 Table: users

Stores manager/dispatcher accounts.

Field	Type	Req.	Description
id	UUID	✓	Primary key
name	VARCHAR	✓	Full display name
email	VARCHAR	✓	Unique login email
password_hash	VARCHAR	✓	Bcrypt hashed password
role	ENUM	✓	admin | dispatcher | viewer
created_at	TIMESTAMP	✓	Auto-set on insert

2.2 Table: workers

All registered site workers. No fixed vehicle — vehicles assigned per dispatch.

Field	Type	Req.	Description
id	UUID	✓	Primary key
name	VARCHAR	✓	Full name
employee_id	VARCHAR	✓	e.g. EMP-001, must be unique
phone	VARCHAR		Contact number
is_active	BOOLEAN	✓	Default true. False = soft delete
created_at	TIMESTAMP	✓	Auto-set on insert
deleted_at	TIMESTAMP		Soft delete — null = active

2.3 Table: vehicles

All fleet vehicles. Type determines wage rate.

Field	Type	Req.	Description
id	UUID	✓	Primary key
plate_number	VARCHAR	✓	e.g. KL-3301, unique
vehicle_type	ENUM	✓	truck | pickup | minivan
status	ENUM	✓	working | maintenance | not_working

Field	Type	Req.	Description
is_active	BOOLEAN	✓	Default true. False = soft delete
created_at	TIMESTAMP	✓	Auto-set on insert
deleted_at	TIMESTAMP		Soft delete

2.4 Table: wage_rates

Configurable wage per vehicle type and distance. Allows future rate changes without code updates.

Field	Type	Req.	Description
id	UUID	✓	Primary key
vehicle_type	ENUM	✓	truck | pickup | minivan
rate_per_run	DECIMAL	✓	RM amount paid per run
effective_from	DATE	✓	Rate valid from this date
notes	TEXT		Optional — e.g. "Revised April 2026"

NOTE — Wage Calculation
Wage = wage_rates.rate_per_run (for the vehicle type used on that assignment).
The frontend auto-calculates this from the rate table. The backend must validate on save.
If rates change in future, old assignments retain their original saved wage — never recalculate.

2.5 Table: job_batches

Each time a dispatcher clicks SUBMIT / LOCK ASSIGNMENTS, a batch is created. All assignments in that dispatch cycle belong to one batch.

Field	Type	Req.	Description
id	UUID	✓	Primary key
created_by	UUID	✓	FK → users.id
route_from	VARCHAR	✓	Start point name
route_to	VARCHAR	✓	Destination name
distance_km	DECIMAL	✓	Distance for this batch
est_hours	DECIMAL		Estimated travel hours
status	ENUM	✓	open | locked | completed
created_at	TIMESTAM	✓	When batch was created

Field	Type	Req.	Description
	P		
locked_at	TIMESTAMP		When SUBMIT was pressed

2.6 Table: assignments

Each row = one worker dispatched for one batch. Vehicle is chosen at assign time — not pre-fixed.

Field	Type	Req.	Description
id	UUID	✓	Primary key
batch_id	UUID	✓	FK → job_batches.id
worker_id	UUID	✓	FK → workers.id
vehicle_id	UUID	✓	FK → vehicles.id (chosen at assign time)
wage_per_run	DECIMAL	✓	Snapshot of rate at time of assignment
runs_completed	INTEGER	✓	Default 0. Updated on return.
total_earned	DECIMAL	✓	Computed: runs_completed × wage_per_run
return_status	ENUM	✓	pending | returned | issue | reassigned
issue_reason	TEXT		Filled if return_status = issue
assigned_at	TIMESTAMP	✓	When assignment was created
returned_at	TIMESTAMP		When worker marked returned

2.7 Table: worker_payments

Tracks each payment made to a worker. Multiple partial payments are supported.

Field	Type	Req.	Description
id	UUID	✓	Primary key
worker_id	UUID	✓	FK → workers.id
amount	DECIMAL	✓	Amount paid in this transaction
paid_by	UUID	✓	FK → users.id (who processed payment)
notes	TEXT		Optional — e.g. date range covered
paid_at	TIMESTAMP	✓	When payment was recorded

2.8 Table: vehicle_costs

All expenses logged against a vehicle — fuel, maintenance, other.

Field	Type	Req.	Description
id	UUID	✓	Primary key
vehicle_id	UUID	✓	FK → vehicles.id
cost_type	ENUM	✓	fuel | maintenance | other
amount	DECIMAL	✓	Cost in RM
note	TEXT		e.g. "Oil change 15 Mar"
logged_by	UUID	✓	FK → users.id
logged_at	TIMESTAMP	✓	Auto-set on insert

3. API Endpoints

Base URL: /api/v1 All responses: { "success": true/false, "data": {}, "error": "" }

Authentication: Bearer JWT token in Authorization header for all protected routes.

3.1 Auth

Method	Endpoint	Description	Auth
POST	/auth/login	Login with email + password. Returns JWT token.	Public
POST	/auth/logout	Invalidate session (optional if stateless JWT).	JWT
GET	/auth/me	Get current logged-in user profile.	JWT

3.2 Workers

Method	Endpoint	Description	Auth
GET	/workers	List all active workers.	JWT
GET	/workers/:id	Get single worker with assignment history.	JWT
POST	/workers	Create new worker.	JWT Admin

Method	Endpoint	Description	Auth
PUT	/workers/:id	Update worker name, employee ID, phone.	JWT Admin
DELETE	/workers/:id	Soft delete worker (sets is_active = false).	JWT Admin
GET	/workers/:id/finance	Earnings, expenses, paid, remaining balance.	JWT
GET	/workers/:id/history	Paginated assignment + payment history.	JWT

3.3 Vehicles

Method	Endpoint	Description	Auth
GET	/vehicles	List all vehicles. Query: ?type=truck&status=working	JWT
GET	/vehicles/:id	Vehicle detail: metrics, costs, trip history.	JWT
POST	/vehicles	Register new vehicle.	JWT Admin
PUT	/vehicles/:id	Update plate, type, or status.	JWT Admin
DELETE	/vehicles/:id	Soft delete vehicle.	JWT Admin
GET	/vehicles/available	Available (working, unassigned) vehicles. Query: ?type=	JWT
POST	/vehicles/:id/costs	Log a cost (fuel / maintenance / other).	JWT
GET	/vehicles/:id/costs	List all costs for a vehicle.	JWT

3.4 Wage Rates

Method	Endpoint	Description	Auth
GET	/wage-rates	Get current rates per vehicle type.	JWT
POST	/wage-rates	Add a new rate entry (new effective date).	JWT Admin

3.5 Job Batches (Dispatch)

Method	Endpoint	Description	Auth
GET	/batches	List all batches. Query: ?	JWT

Method	Endpoint	Description	Auth
		status=open&date=	
GET	/batches/:id	Batch detail with all assignments.	JWT
POST	/batches	Create a new open batch (job setup).	JWT
PUT	/batches/:id/lock	SUBMIT / LOCK — locks batch, pushes to P2.	JWT
PUT	/batches/:id	Update batch route/distance while still open.	JWT

3.6 Assignments

Method	Endpoint	Description	Auth
GET	/batches/:id/assignments	List all assignments in a batch.	JWT
POST	/batches/:id/assignments	Assign a worker + vehicle to a batch.	JWT
PUT	/assignments/:id	Reassign — update vehicle_id and recalc wage.	JWT
PUT	/assignments/:id/return	Mark worker returned. Body: { runs_completed }.	JWT
PUT	/assignments/:id/issue	Flag an issue. Body: { issue_reason }.	JWT

3.7 Payments

Method	Endpoint	Description	Auth
GET	/workers/:id/payments	List all payments made to a worker.	JWT
POST	/workers/:id/payments	Record a payment. Body: { amount, notes }.	JWT

3.8 Dashboard

Method	Endpoint	Description	Auth
GET	/dashboard/summary	Working vehicles, total expenses, runs today, workers on duty.	JWT

Method	Endpoint	Description	Auth
GET	/dashboard/activity	Last 15 activity log entries.	JWT

4. Request & Response Examples

4.1 POST /assignments — Assign a Worker

Request Body

```
POST /api/v1/batches/{batch_id}/assignments
{
  "worker_id":    "uuid-of-worker",
  "vehicle_id":   "uuid-of-vehicle",
  "wage_per_run": 85.00
}
```

Success Response (201 Created)

```
{
  "success": true,
  "data": {
    "id":             "uuid-of-assignment",
    "batch_id":       "uuid-of-batch",
    "worker_id":      "uuid-of-worker",
    "vehicle_id":     "uuid-of-vehicle",
    "wage_per_run":   85.00,
    "runs_completed": 0,
    "total_earned":   0,
    "return_status":  "pending",
    "assigned_at":    "2026-04-01T09:00:00Z"
  }
}
```

4.2 PUT /assignments/:id/return — Worker Returned

Request Body

```
{  "runs_completed": 3  }
```

Success Response (200 OK)

```
{
  "success": true,
  "data": {
```

```
      "id":             "uuid-of-assignment",
      "runs_completed": 3,
      "total_earned":   255.00,
      "return_status":  "returned",
      "returned_at":    "2026-04-01T14:30:00Z"
    }
  }
```

4.3 GET /workers/:id/finance

Success Response (200 OK)

```
  {
    "success": true,
    "data": {
      "worker_id":    "uuid",
      "worker_name":  "Ahmad Raza",
      "total_earned": 1275.00,
      "total_paid":   900.00,
      "remaining":    375.00,
      "total_runs":   15,
      "expenses":     120.00
    }
  }
```

5. Business Logic Rules

5.1 Wage Calculation

- Wage is determined by vehicle TYPE at the time of assignment.
- Rate is fetched from wage_rates where effective_from ≤ today, ordered by effective_from DESC, LIMIT 1.
- Frontend pre-calculates and shows the wage. Backend MUST validate the wage_per_run on save.
- Once an assignment is saved, the wage_per_run must NEVER be retroactively changed.
- total_earned = runs_completed × wage_per_run — computed on every return update.

5.2 Vehicle Assignment Rules

- A worker has NO fixed vehicle. Vehicle is chosen fresh at every dispatch.
- A vehicle can only be assigned to ONE active assignment at a time.
- Only vehicles with status = "working" appear in the assign modal dropdown.

- If a vehicle is reassigned mid-batch (REASSIGN flow), the old assignment must be updated before creating the new one.
- Reassigning does NOT create a new assignment row — it updates vehicle_id and wage_per_run on the existing row.

5.3 Batch / Dispatch Flow

1. Dispatcher fills Job Setup (route, distance, time) → create batch (status = open).
2. Dispatcher assigns workers one by one → POST /assignments for each.
3. SUBMIT / LOCK ASSIGNMENTS pressed → PUT /batches/:id/lock → status = locked.
4. P2 (Work Status) loads — all assignments for the locked batch displayed.
5. Dispatcher marks each worker returned (with runs count) or raises an issue.
6. When all returned → batch status = completed.

5.4 Payment Logic

- Payments are additive — multiple partial payments per worker are supported.
- remaining = SUM(assignments.total_earned) - SUM(payments.amount)
- Payments are NOT tied to a specific assignment — they cover the worker's running balance.
- A negative remaining balance is technically possible (overpayment) — the UI should warn but not block.

6. Error Handling

HTTP Code	Error Key	When to Use
400	VALIDATION_ERROR	Missing required fields or invalid values in request body
401	UNAUTHORIZED	No JWT token or token expired
403	FORBIDDEN	Valid JWT but insufficient role (e.g. viewer trying to delete)
404	NOT_FOUND	Worker, vehicle, assignment, or batch ID does not exist
409	CONFLICT	Vehicle already assigned, duplicate employee ID, etc.
422	WAGE_MISMATCH	Frontend wage does not match backend-calculated wage
500	INTERNAL_ERROR	Unexpected server error — log and return generic message

Standard error response shape

```
{
  "success": false,
```

```
  "error": {
    "code":    "CONFLICT",
    "message": "Vehicle KL-3301 is already assigned in this batch."
  }
}
```

7. Authentication & Security

7.1 JWT Setup

- On login, issue a signed JWT with payload: { userId, role, iat, exp }.
- Token expiry: 8 hours (one full workday shift).
- Store secret in environment variable JWT_SECRET — never hardcode.
- All protected routes use a middleware that verifies the token before passing to the handler.

7.2 Roles

admin	Full access — create/delete workers and vehicles, manage rates, process payments
dispatcher	Can create batches, assign workers, mark returns, log vehicle costs
viewer	Read-only — dashboards, reports, finance viewing only

7.3 Input Validation Rules

- All string inputs: sanitize (trim whitespace, strip HTML tags).
- Plate numbers: uppercase, validate format with regex.
- Amounts: must be positive numbers. Reject zero or negative.
- Enums: validate against allowed values server-side — never trust frontend.
- UUIDs: validate format on all FK references before querying.

8. Recommended Build Order

Build in this order to always have a working system at each stage.

Phase 1 — Foundation (Week 1)

1. Set up project (Node/Express or FastAPI), connect PostgreSQL.

2. Run migrations to create all 7 tables.
3. Seed: 3 test users, 5 workers, 9 vehicles, wage rates.
4. Build Auth endpoints (login, /me).
5. Test auth flow end-to-end with Postman.

Phase 2 — Core CRUD (Week 2)

6. Workers CRUD (GET list, GET by id, POST, PUT, DELETE soft).
7. Vehicles CRUD + available endpoint.
8. Wage rates GET + POST.
9. Vehicle costs POST + GET.

Phase 3 — Dispatch Flow (Week 3)

10. Job batches — create, list, get, lock.
11. Assignments — create (assign), update (reassign), return, issue.
12. Validate wage on assignment save.
13. Test full P1 → P2 flow end-to-end.

Phase 4 — Finance & Dashboard (Week 4)

14. Worker finance endpoint (earnings, paid, remaining).
15. Payments POST + GET.
16. Dashboard summary endpoint.
17. Activity log (simple event table or derived from assignment events).
18. Connect frontend to live API, replace all mock data.

9. Environment & Project Structure

9.1 Environment Variables (.env)

```
DATABASE_URL=postgresql://user:password@localhost:5432/quarryos
JWT_SECRET=your-super-secret-key-here
JWT_EXPIRES_IN=8h
PORT=3000
NODE_ENV=development
```

9.2 Suggested Folder Structure (Node/Express)

```
quarryos-backend/
  ├── prisma/
```

```
│   ├── schema.prisma          ← DB models
│   └── seed.js                ← Test data
├── src/
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── workers.routes.js
│   │   ├── vehicles.routes.js
│   │   ├── batches.routes.js
│   │   ├── assignments.routes.js
│   │   ├── payments.routes.js
│   │   └── dashboard.routes.js
│   ├── controllers/           ← Business logic per resource
│   ├── middleware/
│   │   ├── auth.js            ← JWT verify
│   │   └── validate.js        ← Input validation
│   ├── utils/
│   │   └── wageCalc.js        ← Wage calculation helper
│   └── app.js                 ← Express setup
├── .env
├── package.json
└── README.md
```

10. Testing Checklist

Before connecting the frontend, verify each of these with Postman or a test runner:

Authentication

- Login returns a valid JWT token.
- Protected route with no token returns 401.
- Protected route with expired token returns 401.
- Viewer role cannot delete a worker (returns 403).

Assignment & Wage

- Assigning a maintenance vehicle returns 409 CONFLICT.
- Assigning the same vehicle twice in one batch returns 409 CONFLICT.
- Wage mismatch between request and backend calculation returns 422.
- Reassigning updates vehicle_id and wage_per_run on the existing assignment row.

Finance

- total_earned updates correctly after marking a worker returned with 3 runs.

- remaining decreases correctly after logging a payment.
- Partial payment does not close the worker balance (remaining stays positive).

Soft Deletes

- Deleted worker does not appear in GET /workers list.
- Historical assignments for deleted worker are still accessible.
- Deleted vehicle does not appear in GET /vehicles/available.